Amortizovaná složitost

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

12.03.2026

CC BY-NC-SA 4.0

Dynamické pole

Motivace

Motivace

Statické pole má pevně danou velikost při vytvoření

Motivace

Statické pole má pevně danou velikost při vytvoření

Co když **nevíme předem**, kolik prvků budeme potřebovat?

Motivace

Statické pole má pevně danou velikost při vytvoření

Co když **nevíme předem**, kolik prvků budeme potřebovat?

Řešení: dynamické pole (nafukovací pole)

Motivace

Statické pole má pevně danou velikost při vytvoření

Co když **nevíme předem**, kolik prvků budeme potřebovat?

Řešení: dynamické pole (nafukovací pole)

- Pole, které se **automaticky zvětšuje** při zaplnění

Princip

Princip

Postup při přidání prvku:

Princip

Postup při přidání prvku:

1. Pokud je v poli místo $\rightarrow$ vlož prvek na konec

Princip

Postup při přidání prvku:

1. Pokud je v poli místo $\rightarrow$ vlož prvek na konec
2. Pokud je pole plné:

Princip

Postup při přidání prvku:

1. Pokud je v poli místo → vlož prvek na konec
2. Pokud je pole plné:
   - Alokuj **nové pole** s dvojnásobnou kapacitou

Princip

Postup při přidání prvku:

1. Pokud je v poli místo → vlož prvek na konec
2. Pokud je pole plné:
   - Alokuj **nové pole** s dvojnásobnou kapacitou
   - Zkopíruj všechny prvky do nového pole

Princip

Postup při přidání prvku:

1. Pokud je v poli místo → vlož prvek na konec
2. Pokud je pole plné:
   - Alokuj **nové pole** s dvojnásobnou kapacitou
   - Zkopíruj všechny prvky do nového pole
   - Uvolni staré pole

Princip

Postup při přidání prvku:

1. Pokud je v poli místo → vlož prvek na konec
2. Pokud je pole plné:
   - Alokuj **nové pole** s dvojnásobnou kapacitou
   - Zkopíruj všechny prvky do nového pole
   - Uvolni staré pole
   - Vlož prvek na konec

Pseudokód

Pseudokód

```
function append(array, element):
    if size == capacity:
        capacity = 2 * capacity
        new_array = allocate(capacity)
        for j = 0, ..., size - 1:
            new_array[j] = array[j]
        free(array)
        array = new_array
    array[size] = element
    size = size + 1
```

Složitost přidání prvku

Složitost přidání prvku

Nejhorší případ (pole je plné):

Složitost přidání prvku

Nejhorší případ (pole je plné): $\mathcal{O}(n)$ – kopírování všech prvků

Složitost přidání prvku

Nejhorší případ (pole je plné): $\mathcal{O}(n)$ – kopírování všech prvků

Nejlepší případ (pole má místo):

Složitost přidání prvku

Nejhorší případ (pole je plné): $\mathcal{O}(n)$ – kopírování všech prvků

Nejlepší případ (pole má místo): $\mathcal{O}(1)$ – pouze vložení

Složitost přidání prvku

Nejhorší případ (pole je plné): $\mathcal{O}(n)$ – kopírování všech prvků

Nejlepší případ (pole má místo): $\mathcal{O}(1)$ – pouze vložení

Ale jak často nastane nejhorší případ?

Složitost přidání prvku

Uvažme n po sobě jdoucích operací `append` (počáteční kapacita = 1):

Složitost přidání prvku

Uvažme n po sobě jdoucích operací `append` (počáteční kapacita = 1):

- **Cena** = 1 (vložení) + případné kopírování všech prvků při realokaci

Složitost přidání prvku

Uvažme n po sobě jdoucích operací `append` (počáteční kapacita = 1):

- **Cena** = 1 (vložení) + případné kopírování všech prvků při realokaci

Operace	1	2	3	4	5	6–8	9
Kapacita	1	2	4	4	8	8	16
Cena	1	1 + 1	1 + 2	1	1 + 4	1	1 + 8

Složitost přidání prvku

Uvažme n po sobě jdoucích operací `append` (počáteční kapacita = 1):

- **Cena** = 1 (vložení) + případné kopírování všech prvků při realokaci

Operace	1	2	3	4	5	6–8	9
Kapacita	1	2	4	4	8	8	16
Cena	1	1 + 1	1 + 2	1	1 + 4	1	1 + 8

Realokace nastává pouze při operaci 2, 3, 5, 9, … ⇒ **stále řidčeji**

Složitost přidání prvku

Operace	1	2	3	4	5	6–8	9	10–16	17	18–32
Kapacita před	1	1	2	4	4	8	8	16	16	32
Realokace		✓	✓		✓		✓		✓	
Kapacita po	1	2	4	4	8	8	16	16	32	32

Složitost přidání prvku

Operace	1	2	3	4	5	6–8	9	10–16	17	18–32
Kapacita před	1	1	2	4	4	8	8	16	16	32
Realokace		✓	✓		✓		✓		✓	
Kapacita po	1	2	4	4	8	8	16	16	32	32

Z 32 operací je pouze **5 drahých** (realokace)

Složitost přidání prvku

Operace	1	2	3	4	5	6–8	9	10–16	17	18–32
Kapacita před	1	1	2	4	4	8	8	16	16	32
Realokace		✓	✓		✓		✓		✓	
Kapacita po	1	2	4	4	8	8	16	16	32	32

Z 32 operací je pouze **5 drahých** (realokace)

Čím více operací, tím menší podíl drahých $\Rightarrow$ amortizovaně $\mathcal{O}(1)$

Amortizovaná složitost

Co je amortizovaná složitost?

Co je amortizovaná složitost?

Amortizovaná složitost je **průměrná cena operace** v nejhorší možné sekvenci operací

Co je amortizovaná složitost?

Amortizovaná složitost je **průměrná cena operace** v nejhorší možné sekvenci operací

- Nejedná se o pravděpodobnostní analýzu

Co je amortizovaná složitost?

Amortizovaná složitost je **průměrná cena operace** v nejhorší možné sekvenci operací

- Nejedná se o pravděpodobnostní analýzu
- Je **garantovaná** pro libovolnou sekvenci operací

Co je amortizovaná složitost?

Amortizovaná složitost je **průměrná cena operace** v nejhorší možné sekvenci operací

- Nejedná se o pravděpodobnostní analýzu
- Je **garantovaná** pro libovolnou sekvenci operací
- Drahé operace se “rozpočítají” přes levné

Co je amortizovaná složitost?

Co je amortizovaná složitost?

Analogie:

Co je amortizovaná složitost?

Analogie: předplatné na MHD

Co je amortizovaná složitost?

Analogie: předplatné na MHD

- Měsíční jízdenka stojí 550 Kč (drahá operace)

Co je amortizovaná složitost?

Analogie: předplatné na MHD

- Měsíční jízdenka stojí 550 Kč (drahá operace)
- Každá další jízda je “zdarma” (levné operace)

Co je amortizovaná složitost?

Analogie: předplatné na MHD

- Měsíční jízdenka stojí 550 Kč (drahá operace)
- Každá další jízda je “zdarma” (levné operace)
- Čím víc jezdíš, tím nižší průměrná cena za jízdu

Co je amortizovaná složitost?

Analogie: předplatné na MHD

- Měsíční jízdenka stojí 550 Kč (drahá operace)
- Každá další jízda je "zdarma" (levné operace)
- Čím víc jezdíš, tím nižší průměrná cena za jízdu

Anglické *"amortize"* = rozpočítat náklady přes více operací

Amortizovaná vs. asymptotická složitost

Amortizovaná vs. asymptotická složitost

Asymptotická složitost ($\mathcal{O}$)

- Určena na základě **nejhoršího možného** běhu **jedné operace**

Amortizovaná vs. asymptotická složitost

Asymptotická složitost ($\mathcal{O}$)

- Určena na základě **nejhoršího možného** běhu **jedné operace**
- Append do dynamického pole: $\mathcal{O}(n)$

Amortizovaná vs. asymptotická složitost

Asymptotická složitost ($\mathcal{O}$)

- Určena na základě **nejhoršího možného** běhu **jedné operace**
- Append do dynamického pole: $\mathcal{O}(n)$

Amortizovaná složitost

- Průměrná složitost operace v **sekvenci** nejhorších možných vstupů

Amortizovaná vs. asymptotická složitost

Asymptotická složitost ($\mathcal{O}$)

- Určena na základě **nejhoršího možného** běhu **jedné operace**
- Append do dynamického pole: $\mathcal{O}(n)$

Amortizovaná složitost

- Průměrná složitost operace v **sekvenci** nejhorších možných vstupů
- Nevyužívá pravděpodobnost $\Rightarrow$ na sekvenci operací je **zaručena**

Amortizovaná vs. asymptotická složitost

Asymptotická složitost ($\mathcal{O}$)

- Určena na základě **nejhoršího možného** běhu **jedné operace**
- Append do dynamického pole: $\mathcal{O}(n)$

Amortizovaná složitost

- Průměrná složitost operace v **sekvenci** nejhorších možných vstupů
- Nevyužívá pravděpodobnost $\Rightarrow$ na sekvenci operací je **zaručena**
- Append do dynamického pole: $\mathcal{O}(1)$ amortizovaně

Odvození vzorce pro geometrickou řadu

Odvození vzorce pro geometrickou řadu

$$S = 1 + q + q^2 + ... + q^k$$

Odvození vzorce pro geometrickou řadu

$$S = 1 + q + q^2 + ... + q^k$$

Vynásobíme obě strany q:

Odvození vzorce pro geometrickou řadu

$$S = 1 + q + q^2 + ... + q^k$$

Vynásobíme obě strany q:

$$q \cdot S = q + q^2 + q^3 + ... + q^{k+1}$$

Odvození vzorce pro geometrickou řadu

$$S = 1 + q + q^2 + ... + q^k$$

Vynásobíme obě strany q:

$$q \cdot S = q + q^2 + q^3 + ... + q^{k+1}$$

Odečteme S od $q \cdot S$:

Odvození vzorce pro geometrickou řadu

$$S = 1 + q + q^2 + ... + q^k$$

Vynásobíme obě strany q:

$$q \cdot S = q + q^2 + q^3 + ... + q^{k+1}$$

Odečteme S od $q \cdot S$:

$$q \cdot S - S = q^{k+1} - 1$$

Odvození vzorce pro geometrickou řadu

$$q \cdot S - S = q^{k+1} - 1$$

Odvození vzorce pro geometrickou řadu

$$q \cdot S - S = q^{k+1} - 1$$

$$S(q-1) = q^{k+1} - 1$$

Odvození vzorce pro geometrickou řadu

$$q \cdot S - S = q^{k+1} - 1$$

$$S(q-1) = q^{k+1} - 1$$

$$S = \frac{q^{k+1} - 1}{q - 1}$$

Odvození vzorce pro geometrickou řadu

$$q \cdot S - S = q^{k+1} - 1$$

$$S(q-1) = q^{k+1} - 1$$

$$S = \frac{q^{k+1} - 1}{q - 1} = 1 + q + q^2 + ... + q^k$$

Odvození vzorce pro geometrickou řadu

$$q \cdot S - S = q^{k+1} - 1$$

$$S(q-1) = q^{k+1} - 1$$

$$S = \frac{q^{k+1} - 1}{q - 1} = 1 + q + q^2 + ... + q^k$$

$$\sum_{i=0}^{k} q^i = \frac{q^{k+1} - 1}{q - 1}$$

Důkaz pro dynamické pole

Důkaz pro dynamické pole

Celková cena n appendů se skládá ze dvou částí:

Důkaz pro dynamické pole

Celková cena n appendů se skládá ze dvou částí:

1. **Vložení prvku**: každý append stojí 1 $\Rightarrow$ celkem n

Důkaz pro dynamické pole

Celková cena n appendů se skládá ze dvou částí:

1. **Vložení prvku**: každý append stojí 1 $\Rightarrow$ celkem n
2. **Kopírování při realokaci**: nastává při kapacitě 1, 2, 4, 8, …

Důkaz pro dynamické pole

Kapacita se zdvojnásobuje: $1, 2, 4, \ldots, 2^k$

Důkaz pro dynamické pole

Kapacita se zdvojnásobuje: $1, 2, 4, \ldots, 2^k$

- Do vzorce pro geometrickou řadu dosadíme $q =$

Důkaz pro dynamické pole

Kapacita se zdvojnásobuje: $1, 2, 4, \ldots, 2^k$

- Do vzorce pro geometrickou řadu dosadíme $q = 2$

Důkaz pro dynamické pole

Kapacita se zdvojnásobuje: $1, 2, 4, \ldots, 2^k$

- Do vzorce pro geometrickou řadu dosadíme $q = 2$

Poslední realokace nastane při kapacitě

Důkaz pro dynamické pole

Kapacita se zdvojnásobuje: $1, 2, 4, \ldots, 2^k$

- Do vzorce pro geometrickou řadu dosadíme $q = 2$

Poslední realokace nastane při kapacitě $2^k \leq n$, tedy $k = \lfloor \log_2 n \rfloor$

Důkaz pro dynamické pole

Kapacita se zdvojnásobuje: $1, 2, 4, \ldots, 2^k$

- Do vzorce pro geometrickou řadu dosadíme $q = 2$

Poslední realokace nastane při kapacitě $2^k \leq n$, tedy $k = \lfloor \log_2 n \rfloor$

Celková cena kopírování prvků při realokacích:

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = 1 + 2 + 4 + 8 + \ldots$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i =$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} =$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} =$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

$$2^{\lfloor \log_2 n \rfloor} \leq$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

$$2^{\lfloor \log_2 n \rfloor} \leq 2^{\log_2 n} =$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

$$2^{\lfloor \log_2 n \rfloor} \leq 2^{\log_2 n} = n$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

$$2^{\lfloor \log_2 n \rfloor} \leq 2^{\log_2 n} = n$$

(z definice: $\log_2 n$ je číslo, na které umocníme 2, abychom dostali n)

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

$$2^{\lfloor \log_2 n \rfloor} \leq 2^{\log_2 n} = n$$

(z definice: $\log_2 n$ je číslo, na které umocníme 2, abychom dostali n)

$$2^{\lfloor \log_2 n \rfloor + 1} - 1 =$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

$$2^{\lfloor \log_2 n \rfloor} \leq 2^{\log_2 n} = n$$

(z definice: $\log_2 n$ je číslo, na které umocníme 2, abychom dostali n)

$$2^{\lfloor \log_2 n \rfloor + 1} - 1 = 2 \cdot \underbrace{2^{\lfloor \log_2 n \rfloor}}_{\leq n} - 1 \leq$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

$$2^{\lfloor \log_2 n \rfloor} \leq 2^{\log_2 n} = n$$

(z definice: $\log_2 n$ je číslo, na které umocníme 2, abychom dostali n)

$$2^{\lfloor \log_2 n \rfloor + 1} - 1 = 2 \cdot \underbrace{2^{\lfloor \log_2 n \rfloor}}_{\leq n} - 1 \leq 2n - 1 <$$

Důkaz pro dynamické pole

$$\sum_{i=0}^{\lfloor \log_2 n \rfloor} 2^i = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{2 - 1} = \frac{2^{\lfloor \log_2 n \rfloor + 1} - 1}{1} = 2^{\lfloor \log_2 n \rfloor + 1} - 1$$

$$2^{\lfloor \log_2 n \rfloor} \leq 2^{\log_2 n} = n$$

(z definice: $\log_2 n$ je číslo, na které umocníme 2, abychom dostali n)

$$2^{\lfloor \log_2 n \rfloor + 1} - 1 = 2 \cdot \underbrace{2^{\lfloor \log_2 n \rfloor}}_{\leq n} - 1 \leq 2n - 1 < 2n$$

Důkaz pro dynamické pole

Celková cena n appendů:

Důkaz pro dynamické pole

Celková cena n appendů:

$$\underbrace{n}_{\text{vložení}} + \underbrace{2n}_{\text{kopírování}} =$$

Důkaz pro dynamické pole

Celková cena n appendů:

$$\underbrace{n}_{\text{vložení}} + \underbrace{2n}_{\text{kopírování}} = 3n$$

Důkaz pro dynamické pole

Celková cena n appendů:

$$\underbrace{n}_{\text{vložení}} + \underbrace{2n}_{\text{kopírování}} = 3n$$

Amortizovaná cena jednoho appendu:

Důkaz pro dynamické pole

Celková cena n appendů:

$$\underbrace{n}_{\text{vložení}} + \underbrace{2n}_{\text{kopírování}} = 3n$$

Amortizovaná cena jednoho appendu:

$$\frac{3n}{n} =$$

Důkaz pro dynamické pole

Celková cena n appendů:

$$\underbrace{n}_{\text{vložení}} + \underbrace{2n}_{\text{kopírování}} = 3n$$

Amortizovaná cena jednoho appendu:

$$\frac{3n}{n} = 3 \in$$

Důkaz pro dynamické pole

Celková cena n appendů:

$$\underbrace{n}_{\text{vložení}} + \underbrace{2n}_{\text{kopírování}} = 3n$$

Amortizovaná cena jednoho appendu:

$$\frac{3n}{n} = 3 \in \mathcal{O}(1)$$

Měření v praxi

Měření v praxi

Porovnání dvou strategií zvětšování pole:

Měření v praxi

Porovnání dvou strategií zvětšování pole:

Počet prvků	realloc každý krok	zdvojnásobení	poměr
1 000 000	0,041 s	0,006 s	7×
10 000 000	0,197 s	0,031 s	6×
50 000 000	0,949 s	0,128 s	7×
100 000 000	2,951 s	0,269 s	11×

Měření v praxi

Porovnání dvou strategií zvětšování pole:

Počet prvků	realloc každý krok	zdvojnásobení	poměr
1 000 000	0,041 s	0,006 s	7×
10 000 000	0,197 s	0,031 s	6×
50 000 000	0,949 s	0,128 s	7×
100 000 000	2,951 s	0,269 s	11×

Zdvojnásobování kapacity je v praxi **výrazně rychlejší**